Dataset and Preprocessing

We developed a reproducible data preparation pipeline using the CCPD2019 dataset. We selected 5,000 images through proportional stratified sampling based on license plate height and two filename-encoded tilt values, preserving their approximate distribution in the original dataset.

The dataset was divided into 3,500 training, 700 validation, and 800 test images using seed 42. Stratified splitting was performed before resizing. Plate-size categories were defined using training-set height thresholds: small (≤83 px), medium (83–112 px), and large (>112 px).

Each image was generated at three scales (1.0, 0.75, and 0.5), producing 15,000 image variants. We converted original CCPD bounding box annotations into normalized YOLO labels and generated plate crops with corresponding text labels for OCR training and evaluation.

For OCR preprocessing experiments, we implemented three independent modes: no preprocessing, grayscale with CLAHE contrast enhancement, and Canny edge detection. The primary experiments used original-scale images, while the downscaled variants supported scale-robustness evaluation.

Sources: Data preparation notebook; preprocessing module; repository.

Deleted: SHA-256 checks identified eight duplicate photographs, which were replaced with unique images from the same sampling groups

Commented [1]: Надо будет раньше указать что мы скейлили картинки

Deleted: , ensuring that all variants of each source image remained in the same split and preventing cross-scale data leakage

Deleted: Bounding boxes and license plate text were extracted from CCPD filename annotations.

Deleted: the bounding boxes

Deleted: ground-truth

Deleted: Metadata files record image paths, split assignments, scales, plate sizes, tilt values, and annotations.

Deleted: Data integrity checks verified image readability, annotation validity, duplicate hashes, and split consistency. The preparation notebook, dataset manifests, and configuration files were provided to ensure reproducibility.

CRNN-based plate recognition

We replaced the frozen EasyOCR baseline with a plate-specific convolutional recurrent neural network (CRNN), trained from scratch. Each crop is converted to grayscale, resized to fit a 48 × 256 canvas while preserving its aspect ratio, padded with black pixels, and normalized to [-1, 1]. Four convolutional blocks (32/64/128/128 channels, each with batch normalization, ReLU and max pooling) produce a 16-step feature sequence. A linear projection feeds two bidirectional LSTM layers with 256 hidden units per direction. The classifier predicts 58 training-set characters plus the CTC blank symbol. The model has 2,999,419 parameters. CTC loss learns the image-to-text alignment without individual character boxes; inference uses greedy decoding, collapsing repeated labels and removing blanks.

Training used 3,500 original-scale training crops and 700 validation crops, without augmentation. We ran 50 epochs with Adam (learning rate 0.001, batch size 32, seed 42). The canonical checkpoint was selected at epoch 39 by validation character accuracy; training took 4,154.9 s (about 69 minutes). The alphabet was built from training labels only. Validation also selected the optional preprocessing and confidence threshold before testing: no additional preprocessing and a mean character confidence threshold of 0.50.

On 800 test crops, CRNN achieved 0.7593 character accuracy and recognized 267 plates exactly, versus 0.3474 and 0/800 for EasyOCR. With the frozen YOLO detector, CRNN achieved 0.6710 and 148/800; with fine-tuned YOLO, these increased to 0.7661 and 296/800. CRNN inference took about 5.7 ms per crop (median, Apple M3, one CPU thread).

Large plates and the 10-19-degree tilt bin remained the weakest recognition groups: OCR-only character accuracy was 0.5658 and 0.5193, respectively. The latter contains only 63 test samples. Fixed input resolution and limited coverage of difficult examples are plausible contributors; larger inputs and size/tilt-balanced training are next steps.

Sources: CRNN implementation; training; canonical results (2 November 2026).

Commented [5]: не важно, на каком железе тренили

Deleted:)

Deleted: and deterministic, single-threaded CPU execution

Commented [6]: nice

Commented [7]: Вообще без понятия что это значит...

Deleted: (one minus normalized edit distance, averaged over all samples)

Commented [8]: Может быть таблицой

Deleted: EasyOCR figures are historical frozen measurements, rather than same-environment reruns.

Commented [10]: Это должно быть в секции Weakness / Limitations

YOLO11s License Plate Detection

We used YOLO11s to locate license plates before recognition. The detector predicts bounding boxes for a single class: license plate. For each image, the highest-confidence box is cropped and passed to the CRNN. We compared two versions of the same detector: a frozen baseline using the MorseTechLab checkpoint, which was already trained to detect license plates, and a version fine-tuned on our CCPD2019 subset.

We fine-tuned the model on 3,500 original-scale training images using a 640-pixel input size, a batch size of 16, and the AdamW optimizer with a learning rate of 0.002. We selected the checkpoint with the highest mAP@0.5 on the 700-image validation split. The best checkpoint came from epoch 4, and training stopped after 24 epochs because validation mAP@0.5 did not improve for 20 consecutive epochs.
On the original-scale test images, fine-tuning increased mAP@0.5 from 0.9073 to 0.9949 and F1 from 0.9167 to 0.9714. The main improvement was recall, which rose from 0.8938 to 0.9975, while precision increased slightly from 0.9408 to 0.9466. The fine-tuned detector also achieved a reported mAP@0.5 of 0.9949 on the 0.75× and 0.5× versions of the same test images without retraining.

Formatted: Heading 2

Commented [11]: Мы использовали вполне конкретную модель для бейслайна, нужна ссылка

Commented [12]: Как будто это должно быть первой строчкой

Deleted: Keeping the architecture the same allowed us to measure the effect of adapting the detector to our data.

Deleted: The test split was held out from training and checkpoint selection.¶

Deleted: 800

Commented [15]: Может быть таблицой + пояснения к ней

Deleted: We used the fine-tuned detector in the final pipeline because it missed fewer plates, giving the recognizer a plate crop in more images.